Days 5–6 KYC Lab – understanding guide

A study guide to what was done in the KYC vector-maths lab, how it works and why. Read it before the trainer walkthrough. It is for your own understanding and is not part of the submission. The project is in `training/week2/day5-6/kyc-vector-math-lab-starter/`.

---

1. The lab in one page

The business problem. A bank's KYC (know your customer) remediation team has a backlog of customer records. Analysts write free-text notes such as "*residence bill expired last year, new one missing*" or "*passport spelling differs from the form*". Today a person reads every note, decides what remediation it needs, and tries to notice by eye when the same customer was entered twice.

The helper does two jobs, using only dot product, cosine similarity and Euclidean distance:

- **Job A – classify:** put each analyst note into one of 5 remediation categories (proof of address, ID mismatch, source of funds, high-risk review, periodic refresh), or MANUAL_REVIEW if the note is empty or unclear.
- **Job B – detect duplicates:** find customer profiles that are probably the same person entered twice, and send every pair to a human. **Nothing is ever merged automatically.**

What the starter was. A half-finished, partly wrong helper. The task was to run it and observe, list every inconsistency, write the 6 missing pieces, fix everything one at a time, prove it end to end, then push to GitLab and walk the trainer through it.

Status.

"Done means" (from the brief)	Status
All tests pass, including the ones you wrote	**46/46** (starter: 11 of 14 failing or erroring)
`check_by_hand.py` says every answer matches	**9/9**
`run_pipeline.py` runs end to end and writes 3 files	Yes
No raw ID in any output or on screen	Yes, and enforced by a guard
`inconsistency_log.md` lists everything, fixed or explained	**20** (17 fixed, 3 explained)
README note: what changed + one limitation	Yes (end of README)
Push to GitLab (branch `your-name-days5-6`) + merge request	**You**
8-minute walkthrough	**You** (section 9)

---

2. The maths in plain words

Function	Formula	Meaning
dot product	a1·b1 + a2·b2 + …	Multiply matching positions and add up
norm (length)	$\sqrt{(a1^2 + a2^2 + \ldots)}$	How long the arrow is
normalize	every number ÷ length	Same direction, length exactly 1. A zero vector has no direction, so it is an error
cosine similarity	dot / (norm · norm)	Do the arrows point the same way? 1 = same, 0 = nothing in common
Euclidean distance	$\sqrt{((a1-b1)^2 + \ldots)}$	How far apart the arrow tips are. 0 = identical

Cosine vs distance (worksheet question 5). P = [3,4,0,0] and Q = [6,8,0,0], where Q is P doubled. Cosine = **1.0** (same mix of checks) but distance = **5.0** (Q simply has more of everything). To decide "same kind of customer", use cosine: it compares shape, not size.

The link (why_cosine experiment 3). For unit-length vectors, $distance^2 = 2 - 2 \times cosine$. The lab shows 0.045 = 0.045.

Cosine of 0 (worksheet question 6). P = [3,4,0,0] (only ID and address checks) and S = [0,0,5,12] (only funds checks and risk flags) share nothing: cosine **0.0**, distance **13.928**. For an analyst, S is a completely different kind of case.

---

3. How the embedder turns text into numbers

`embedder.py` builds a **28-number vector**:

- **12 concept dimensions** from `data/concepts.json`: id_document, name_mismatch, address_proof, stale_expired, missing_absent, funds_source, unexplained, pep_high_risk, cash_business, offshore, dormant_refresh, photo_quality. A word in a concept's list adds **1** to that dimension.
- **16 hashed buckets** for every other word (names, streets). Each such word adds **0.5** (the `bucket_weight` in config.json) to one bucket chosen by a checksum of the word.
- Before that, the text is lower-cased, split into words, stop words ("the", "for") are dropped, and endings are lightly stemmed ("expired" → "expir", "years" → "year").

---

4. Job A step by step (real numbers, record K011)

Note: **"Utility bill for the residence expired last year, new one missing"**

1. **Tokens:** utility, bill, residence, expir, last, year, new, one, miss
2. **Concepts hit:** address_proof **3** (utility, bill, residence), stale_expired **1** (expired), missing_absent **1** (missing), dormant_refresh **1** ("year" is in that list as "years"); "last", "new" and "one" go into hashed buckets (0.5 each).
3. **Category vector** for PROOF_OF_ADDRESS (from its description "Address proof missing or expired: utility bill or rent lease for the residence..."): address_proof **6**, stale_expired 1, missing_absent 2, plus buckets.
4. **Cosine:**
5. dot = 3×6 + 1×1 + 1×2 = **21** (the buckets do not overlap)
6. norm(note) = **3.571**, norm(category) = **6.461**
7. cosine = 21 / (3.571 × 6.461) = **0.910**
8. **All five categories:** PROOF_OF_ADDRESS **0.910**, PERIODIC_REFRESH 0.294, HIGH_RISK_REVIEW 0.196, ID_MISMATCH 0.012, SOURCE_OF_FUNDS 0.000. The **highest** wins, and 0.910 ≥ `min_note_score` 0.30, so the note goes to PROOF_OF_ADDRESS with MEDIUM priority and the action "request a fresh address proof".
9. An **empty** note (K020) gives the all-zero vector, so it goes to MANUAL_REVIEW with HIGH priority.

In the starter, step 5 used `min()`: the **lowest** score won (SOURCE_OF_FUNDS 0.000), which is below 0.30, so **every** note ended in MANUAL_REVIEW. Result: 1/20 correct. After the fix: 20/20.

---

5. Job B step by step (real numbers, K003 vs K017)

Profiles are built from name + address + occupation:

- K003: "Ananya Menon Nair · Flat 12 Lotus Apartments Kothrud · chartered accountant"
- K017: "Ananya R Menon · 12 Lotus Apartments Kothrud Pune · chartered accountant"

None of these words are KYC concepts, so the whole vector lives in the hashed buckets. The two vectors differ in only one bucket (0.5 apart): **cosine 0.976, distance 0.5**. That is ≥ 0.85 (`duplicate_cosine`), so the pair is **PROBABLE_DUPLICATE_HUMAN_REVIEW**. A human decides; nothing is merged.

Decision	Rule (config.json)
PROBABLE duplicate	cosine ≥ 0.85
POSSIBLE duplicate	0.70 ≤ cosine < 0.85
not reported	cosine < 0.70

Final result: probable K003/K017 (0.976) and K001/K018 (0.964); possible K002/K019 (0.825, a real duplicate), and 4 pairs that are *not* duplicates (K005/K015, K007/K020 0.77, K004/K016 0.751, K011/K013 0.707). Every one goes to a human. This is the limitation in section 8.

Every pair is compared **once** (j starts at i + 1). In the starter, j started at i, so each record was compared with itself (cosine 1.0) and reported as its own duplicate: 20 fake pairs, precision 0.095.

---

6. The inconsistencies, grouped the way the brief asks

Kind (brief section 9)	What was found (number in `inconsistency_log.md`)
Maths	`norm()` had no square root (#1); `euclidean_distance()` was really Manhattan distance (#2); `classify_note()` used min instead of max (#10); `find_duplicates()` compared records with themselves (#11)
Code vs documentation	zero vector gave ZeroDivisionError, not ValueError (#3); cosine range −1…1 vs "0 to 1" (#4, explained); no lower-casing despite the docstring (#7); "32 buckets" vs 16 (#8); why_cosine printed the wrong vectors (#17); README "everything should pass" (#19, explained)
Tests vs maths	expected cosine([1,0],[1,1]) = 0.5; the right value is 0.7071 (#5)
Data vs data / docs	worksheet R [4,3,0,0] but data [4,3,0,1] (#6); duplicate record ID K013 (#14); date 27/05/2025 not ISO (#15); status "ACTIVE" (#16); unreachable lexicon word "proof-of-identity" (#9); "photo" in two concepts (#18, explained)
Configuration	`review_cosine` 0.9, above `duplicate_cosine` 0.85 and against the documented 0.70 (#13)
Privacy and safety	the "masked_id" column held the raw ID: the rule was stated but not enforced (#12)
Missing	tests/test_pipeline.py and the zero-vector test (#20)

Why order mattered ("some problems hide others"). Nothing could run until `dot()` existed. Once the maths worked, the wrong test value and the worksheet data error appeared. Once classification worked, the self-pairs became visible. Once those were gone, the threshold problem showed (K002/K019 was missing). The duplicate K013 only became obvious when a pair "K011–K013" appeared and nobody could tell which K013 it was. Every step is saved in `evidence/`, 00 to 10.

---

7. The missing pieces (what each one does)

Id	File	What was written
M1	`vector_math.py` `dot()`	`sum(x * y for x, y in zip(a, b))`, after the same-length check
M2	`vector_math.py` `normalize()`	divide each number by `norm(a)`; raise `ValueError` if the length is 0

Id	File	What was written
M3	`kyc_pipeline.py` `mask_id()`	split at the last `-`; keep the prefix and the last 3 characters, replace the rest with `*`; a number of 3 characters or fewer is fully masked (showing it would reveal the whole ID). `SYNID-48213` → `SYNID-**213`
M4	`kyc_pipeline.py` `write_csv()`	create the folder, header from the dict keys, one row per dict (`csv.DictWriter`)
M5	`run_pipeline.py` `main()`	classify → duplicates → evaluate → print two tables → write `remediation_queue.csv`, `duplicate_pairs.csv`, `summary.md`. **Privacy guard:** checks the screen output and every file for any raw ID; if one is found, it deletes the outputs and stops
M6	`tests/test_pipeline.py`	30 tests: masking (6), classification (4), duplicates (5), end-to-end quality and output files (7), data hygiene (8: unique IDs, ISO dates, status values, valid categories, gold pairs exist, synthetic IDs, consistent thresholds)

One bug in M5 itself was caught by its own test: the "Wrote:" line crashed when the output folder was outside the project. It is fixed. This is the brief's point that "the tests and the checker are the judges, not Claude Code".

---

8. The limitation (for the README and the walkthrough)

The toy embedder sends every word it doesn't know (names, streets) into only **16 buckets**, so unrelated words collide. Profiles are almost entirely such words, so different people can look alike: 4 of the 5 POSSIBLE pairs are not real duplicates. They go to a human, so it is review work, not a wrong merge. The real duplicates also share the same `id_number`, a much stronger signal that this vector-only design ignores.

Next step: a real embedding model replaces `embedder.py` (the maths and pipeline stay the same), and an exact-ID check runs before the cosine step.

Also worth knowing: masked IDs are not unique (K008 and K013 are both `SYNID-**456`). Always refer to records by `record_id`.

---

9. How to run and demo (about 8 minutes)

```
cd kyc-vector-math-lab-starter
python -m unittest discover -s tests       # 46 tests OK
python check_by_hand.py                    # all answers match
python why_cosine.py                       # four experiments
python run_pipeline.py                     # two tables + output/ files + privacy check
```

Time	What to say or show
0:30	**Problem:** analysts triage KYC notes by hand and miss duplicate customers. **Outcome:** a helper that sorts notes by cosine similarity and sends probable duplicates to a human, with IDs always masked
1:00	**Architecture:** `docs/diagrams/architecture.png`: records → embedder → Job A (highest cosine, weak → manual review) and Job B (each pair once, 0.85 / 0.70) → masked outputs
2:00	**Two inconsistencies and how found:** (1) the test expecting 0.5 for cosine([1,0],[1,1]): worked by hand, it is 1/√2 = 0.707, so the test was wrong; (2) the privacy guard stopping the first run because the "masked" column held raw IDs. Optional third: `min` instead of `max`, 1/20 → 20/20
3:00	**Live:** run the tests, then the pipeline; point at the masked IDs and the HUMAN_REVIEW decisions
1:00	**Limitation:** the 16-bucket collisions and the false possible duplicates; next step a real model plus an exact-ID check

Likely questions

- *Why restore 0.70 instead of choosing your own threshold?* Thresholds belong to the process owner. 0.70 is the documented decision (README, diagram, function default); 0.9 contradicted all three and was even above the "probable" line.
- *How do you know the data was wrong and not your paper answer for R?* The worksheet and the docstring example both use [4,3,0,0]; only the data file differed. Three independent sources agreed.
- *Why mask a 3-character number completely?* Keeping "the last 3" of a 3-character number would show all of it.
- *Why does K020 (empty note) get HIGH priority?* Unknown means a person must look; it must not wait at the bottom of the queue.

Glossary

Term	Meaning
KYC	Know your customer: identity, address and funds checks a bank must keep up to date
Remediation	Fixing a gap in a customer's KYC record
PEP	Politically exposed person: needs enhanced due diligence
Embedding	Text turned into a list of numbers
Hashed bucket	One of 16 extra dimensions that unknown words fall into, picked by a checksum
Precision	Of the pairs flagged probable, how many are real duplicates
Recall	Of the real duplicates, how many were found
Masking	Hiding most characters of an ID: `SYNID-**213`
Gold data	The known right answers, used only to measure quality